

Todo List Application

Anh Khai Phung

Today's agenda		
→	Scope & deliverables	
→	Approach	
→	Design	
→	Running the Project & Seed data	
→	Testing & Verification	
→	Q&A	

→ Scope & deliverables

→ Approach

→ Design

→ Running the Project & Seed data

→ Testing & Verification

→ Q&A



Scope & Deliverables

- A simple to-do list app
- Command-line-based UI
- Supported operations
 - Adding task
 - Removing tasks
 - Listing tasks
 - Classifying tasks
 - Saving list to a file



What is out of scope

- **Complex querying**
 - Is not needed to achieve functional requirements
 - Timespan does not allow for such a system
 - We can design the system to expect such an addition in the future
- **Sorting of tasks and categories**
 - Helps out greatly for querying speed
 - But hard to say what the tasks needed to be sorted by to be most convenient for the user
 - Committing to an organization scheme early might not be great for modifiability
 - Once again, a design problem
- **Atomicity of serialization**
 - The work to actually prove this is hard (staging environments, scripting a crash, etc...)
 - You can follow design patterns, but without proofs the feature is null and void



Approach

1

Scoping the problem

Figure out what concrete features needed to be done and what can be left for later consideration.

2

Initial design work

Produce an initial design that can hit all of the deliverables. Does not have to be perfect. The goal is to guesstimate what methods are needed for the feature to function.

3

Implementation Loop

For each method: write a failing test case, write minimum code to pass, reflect on overall design and alter if needed, repeat.

4

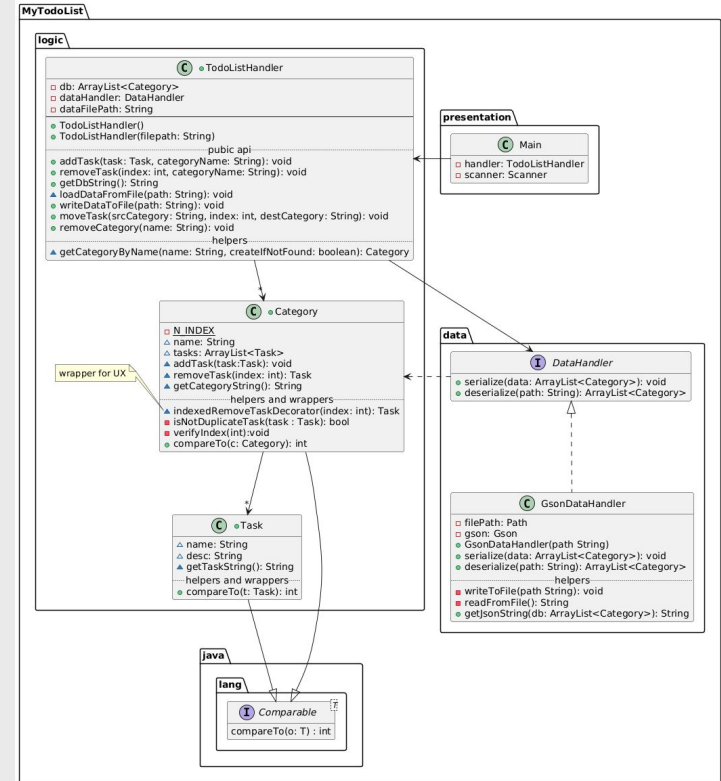
Verification and dry runs

After all features are finished, do some small scale manual tests to verify feature. Even better if there is time to script an automated integration test.



Design

- **Based off of three layers architecture**
- **Let it fail**
 - Invalid operations throws exceptions if something is wrong
 - Easier to maintain and debug
- **Fields in business domain are mostly package private**
 - Easy for testing/verifying internal representations
 - Internal components trust each other inherently when it comes to exposed fields, security can be layered/wrapped on the outer fringe when needed
- **Data handler are designed to be dependency agnostic**
 - Underlying serialization scheme can be swapped fairly easily (toml, protobuf, yaml, binary, etc...)





Alternatives considered

- **Category not being an object (2D array of Tasks or HashMap)**
 - Gives really good querying speed but rigid querying scheme
 - Hard to implement complex querying in the future
 - Hard to modify category in the long run
- **Passing around null Categories**
 - Passing around nulls is easy, but risky considering future maintenance
 - Using a special case object (i.e a default category) solves this problem



Running the project

To run the project, you need

- Maven 3.9.16
- Java 17

After dependencies are installed and set up, navigate to the project root, invoke maven to compile the project and run

```
mvn compile
```

```
# If you are on cmd or bash
```

```
mvn exec:java -Dexec.mainClass="MyTodoList.Main" -Dexec.args="src/main/resources/seedData2.json"
```

```
# If you are on Powershell
```

```
mvn exec:java "-Dexec.mainClass=MyTodoList.Main" "-Dexec.args=src/main/resources/seedData2.json"
```




Seeding Data

Seed data are comes in two files. They can be found in

- src/main/resources/seedData2.json
- src/test/resources/seedData.json

The seed can be applied to the application via command line arguments by filling in **-Dexec.args** (check previous slide)

Note: The application only can take one file path in args[0] as seed. Multiple seeds are not supported and will be ignored.



Testing & Verification

- Test case are located in **src/test**
- They can be run using maven via the command "mvn test" on the terminal from the project root
- Cases should cover most all of the interesting cases in domain logic
- CI with maven is set up for the repository
- Test suite comprises of mostly unit tests, only a few happy path integration tests are written.



Thank you

If you have any questions, feel free to ask!

You can contact me at: **phunga@rose-hulman.edu**